

MODUL PRAKTIKUM KECERDASAN BUATAN

PERTEMUAN 1 – *REPRESENTASI PENGETAHUAN*

Tools : Jupyter notebook

Bahasa Pemrograman : Python

1. Pengenalan Python dan Google Colab

Python¹ didesain oleh Guido van Rossum² pada 1991 sebagai general-purpose language dan bersifat open-source. Python merupakan bahasa pemrograman populer tetapi juga memiliki properti yang sangat baik untuk programmer pemula, sehingga ideal untuk orang yang belum pernah memprogram sebelumnya. Python juga bersifat cross-platform, artinya bisa digunakan pada sistem operasi Windows, Mac OS, dan Linux.

Beberapa kelebihan dari properti tersebut adalah kode yang mudah dibaca, pengetikan dan penggunaan memori yang dinamis. Python adalah bahasa interpreter, sehingga kode dieksekusi segera di konsol Python tanpa memerlukan langkah kompilasi ke bahasa mesin. Selain konsol Python (yang disertakan dengan instalasi Python apa pun), Anda dapat menemukan konsol interaktif lainnya, seperti IPython (Interactive Python)³ dan Google Colab⁴ yang memberi Anda lingkungan yang lebih interaktif untuk mengeksekusi kode Python Anda.

Gambar 1 10 Most In-Demand Data Science Skills in 2021 (<https://towardsdatascience.com/>)

2. Library Dasar Python

Komunitas Python adalah salah satu komunitas pemrograman paling aktif dengan banyaknya pengembang library. Library Python yang paling populer untuk proyek data science adalah NumPy, SciPy, Pandas, Scikit-Learn, dan Matplotlib.

a. NumPy and SciPy: Numeric and Scientific Computation

- Library dasar untuk perhitungan saintifik (scientific computing) dengan Python <https://numpy.org/>
- Alternatif untuk Python List: Numpy Array untuk n dimensi
- Mudah digunakan dan bersifat open source
- Jika library belum terpasang, tuliskan perintah instalasi :
`pip install numpy`
- Kemudian impor
`import numpy as np`

b. SciPy

- SciPy (dibaca “Sigh Pie”) merupakan library yang bersifat open source dan tersedia di <https://www.scipy.org/>
- SciPy dibangun untuk bekerja dengan NumPy array dan menyediakan kumpulan algoritma • numerik , termasuk pemrosesan sinyal , optimasi , statistika , dan library Matplotlib untuk visualisasi data.
- Jika library belum terpasang ,tuliskan perintah instalasi :
`pip install scipy`

c. Pandas

- Pandas (Panel Data) merupakan library populer di Python yang digunakan untuk data structure dan data analysis
- Bersifat open source dan tersedia di <https://pandas.pydata.org/>
- Pandas sangat berkaitan dengan NumPy
- Jika library belum terpasang, tuliskan perintah instalasi :
`pip install pandas`
- Kemudian impor
`import pandas as pd`
- Pandas dapat mengimpor data dari berbagai format: comma separated value (CSV), file tek, Microsoft Excel, database SQL, dan format HDF5

d. Matplotlib

- Matplotlib adalah library Python untuk visualisasi data dengan dua dimensi • Bersifat open source dan tersedia di <https://matplotlib.org/>
- Matplotlib berkaitan dengan NumPy dan Pandas
- Jika library belum terpasang, tuliskan perintah instalasi :
`pip install matplotlib`
- Kemudian impor
`import matplotlib.pyplot as plt`

e. Seaborn

- Seaborn adalah library visualisasi data Python serupa dengan Matplotlib yang menyediakan high level interface untuk menggambar grafik statistika yang menarik dan informatif • Library ini bersifat open source dan tersedia di <https://seaborn.pydata.org/>
- Jika library belum terpasang, tuliskan perintah instalasi :
`pip install seaborn`
- Kemudian impor
`import seaborn as sns`

f. Scikit-learn

- Scikit-learn adalah library untuk mempraktikkan machine learning dan membuat model. Bersifat open source dan tersedia di <https://scikit-learn.org/>
- Scikit-learn diawali dari project SciPy (Scientific Python) yang berisi fungsi-fungsi matematis • Jika library belum terpasang, tuliskan perintah instalasi:
`pip install sklearn`
- Kemudian impor:
`import sklearn`

g. Pydotplus

- Perintah instalasi:
`conda install -c conda-forge pydotplus`

3. Jupyter Notebook

Dengan munculnya aplikasi web, generasi baru IDE untuk bahasa interaktif seperti Python telah dikembangkan. Dimulai di komunitas akademisi dan e-learning, IDE berbasis web dikembangkan dengan mempertimbangkan agar kode dan seluruh environment Anda dapat disimpan di server. Salah satu aplikasi pertama dari jenis WIDE ini dikembangkan oleh William Stein pada awal 2005 menggunakan Python 2.3 sebagai bagian dari perangkat lunak matematika SageMath. Di SageMath, server dapat diatur di pusat, seperti universitas atau sekolah, dan kemudian siswa dapat mengerjakan pekerjaan rumah mereka baik di kelas maupun di rumah. Selain itu, siswa dapat menjalankan semua langkah sebelumnya berulang kali,

dan kemudian mengubah beberapa sel kode tertentu (segmen dokumen yang mungkin berisi kode sumber yang dapat dieksekusi) dan menjalankan operasi lagi. Instruktur juga dapat memiliki akses ke sesi siswa dan meninjau kemajuan atau hasil mereka.

Saat ini, sesi semacam itu disebut notebook dan tidak hanya digunakan di ruang kelas tetapi juga digunakan untuk menunjukkan hasil dalam presentasi atau di dasbor bisnis. Penggunaan notebook adalah

bagian dari pengembangan IPython. Sejak Desember 2011, IPython telah dirilis sebagai versi browser dari konsol interaktifnya, yang disebut notebook IPython, yang menunjukkan hasil eksekusi Python dengan sangat jelas dan ringkas melalui sel baris. Sel dapat berisi konten selain kode. Misalnya, sel markdown yang dapat ditambahkan pada sel untuk menjelaskan algoritma yang dibuat. Dimungkinkan juga untuk menyisipkan gambar plot Matplotlib untuk mengilustrasikan contoh atau bahkan halaman web. Belakangan ini, beberapa jurnal ilmiah mulai menerima notebook untuk menunjukkan hasil eksperimen, dilengkapi dengan kode dan sumber datanya. Dengan cara ini, eksperimen menjadi lengkap dan dapat direplikasi.

Sejak proyek data science berkembang pesat, notebook IPython telah dipisahkan dari perangkat lunak IPython dan sekarang telah menjadi bagian dari proyek yang lebih besar: Jupyter¹⁶. Jupyter (untuk Julia, Python dan R) bertujuan untuk menggunakan kembali WIDE yang sama untuk semua bahasa interpreter ini dan tidak hanya Python. Semua notebook IPython lama diimpor secara otomatis ke versi baru saat dibuka dengan platform Jupyter; tetapi setelah dikonversi ke versi baru, notebook tidak dapat digunakan lagi di versi notebook IPython lama.

4. Representasi Pengetahuan

a. Data Frame

Data frame merupakan tabel/data tabular dengan array dua dimensi yaitu baris dan kolom. Struktur data ini merupakan cara paling standar untuk menyimpan data. Setiap kolom pada data frame merupakan objek dari Series, dan baris terdiri dari elemen yang ada pada Series.

Series merupakan struktur data dasar dalam Pandas. Series diibaratkan sebagai array satu dimensi sama halnya dengan numpy array, hanya bedanya mempunyai index dan index tersebut dapat kita kontrol dari setiap elemen tersebut.

REPRESENTASI DATA FRAME

a. Membuat Data Frame

```
In [35]: import numpy as np
import pandas as pd

In [36]: my_data = np.array([[5.1,3.5,1.4,0.2],[4.9,3.0,1.4,0.2],[4.7,3.2,1.3,0.2],[4.6,3.1,1.5,0.2],[5.0,3.6,1.4,0.2]
])

In [37]: column_names = ['sepal.length','sepal.width','petal.length','petal.width']

In [38]: data_frame = pd.DataFrame(data=my_data, columns=column_names)
print(data_frame)
```

	sepal.length	sepal.width	petal.length	petal.width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

b. Menambahkan Kolom Baru

```
In [39]: data_frame["species"] = 'Iris-setosa'

In [40]: print(data_frame)
```

	sepal.length	sepal.width	petal.length	petal.width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

c. Menentukan Subset

```
In [44]: print("rows 0, 1, and 2 : ")
print(data_frame.head(3), '\n')

print("row 3 : ")
print(data_frame.iloc[[3]], '\n')

print("row 1, 2, and 3 : ")
print(data_frame[1:4], '\n')

print("Column 'species' : ")
print(data_frame["species"])
```

```
rows 0, 1, and 2 :
   sepal.length  sepal.width  petal.length  petal.width  species
0           5.1           3.5           1.4           0.2  Iris-setosa
1           4.9           3.0           1.4           0.2  Iris-setosa
2           4.7           3.2           1.3           0.2  Iris-setosa

row 3 :
   sepal.length  sepal.width  petal.length  petal.width  species
3           4.6           3.1           1.5           0.2  Iris-setosa

row 1, 2, and 3 :
   sepal.length  sepal.width  petal.length  petal.width  species
1           4.9           3.0           1.4           0.2  Iris-setosa
2           4.7           3.2           1.3           0.2  Iris-setosa
3           4.6           3.1           1.5           0.2  Iris-setosa

Column 'species' :
0  Iris-setosa
1  Iris-setosa
2  Iris-setosa
3  Iris-setosa
4  Iris-setosa
Name: species, dtype: object
```

b. Decision Tree

Decision tree adalah alat pendukung dengan struktur seperti pohon yang memodelkan kemungkinan hasil, biaya sumber daya, utilitas, dan kemungkinan konsekuensi.

Decision tree menyediakan cara untuk menyajikan algoritma dengan pernyataan kontrol bersyarat. Mereka termasuk cabang yang mewakili langkah-langkah pengambilan keputusan yang dapat mengarah pada hasil yang menguntungkan. Struktur flowchart mencakup node internal yang mewakili tes atau atribut pada setiap tahap. Setiap cabang mewakili hasil untuk atribut, sedangkan jalur dari daun ke akar mewakili aturan untuk klasifikasi.

Disebut decision tree atau pohon keputusan karena pilihannya bercabang, membentuk struktur yang terlihat seperti pohon. Pohon keputusan dapat dibuat secara vertikal atau horizontal. Pohon keputusan horizontal dari kiri ke kanan dan pohon keputusan vertikal dari atas ke bawah.

REPRESENTASI DECISION TREE

```
In [8]: import pandas as pd
import numpy as np
```

```
In [9]: data = pd.read_csv('Iris.csv')
data.head(10)
```

```
Out[9]:
```

	sepal.length	sepal.width	petal.length	petal.width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

```
In [10]: data.describe()
```

```
Out[10]:
```

	sepal.length	sepal.width	petal.length	petal.width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.854000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.900000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
In [11]: data.isnull().sum()
```

```
Out[11]: sepal.length    0
sepal.width    0
petal.length    0
petal.width    0
species    0
dtype: int64
```

```
In [12]: data[['species', 'sepal.length', 'sepal.width', 'petal.length', 'petal.width']].sample(100)
```

```
Out[12]:
```

	species	sepal.length	sepal.width	petal.length	petal.width
141	Iris-virginica	6.9	3.1	5.1	2.3
134	Iris-virginica	6.1	2.8	5.6	1.4
69	Iris-versicolor	5.6	2.5	3.9	1.1
39	Iris-setosa	5.1	3.4	1.5	0.2
128	Iris-virginica	6.4	2.8	5.6	2.1
...	...	...	...	...	...
28	Iris-setosa	5.2	3.4	1.4	0.2
132	Iris-virginica	6.4	2.8	5.6	2.2
130	Iris-virginica	7.4	2.8	6.1	1.8
83	Iris-versicolor	6.0	2.7	5.1	1.6
116	Iris-virginica	7.7	2.6	6.9	2.3

100 rows x 5 columns

```
In [14]: from sklearn.model_selection import train_test_split
predictors = data[['sepal.length', 'sepal.width', 'petal.length', 'petal.width']]
targets = data.species
x_train, x_test, y_train, y_test = train_test_split(predictors, targets, test_size=.3, random_state=0)
```

```
In [15]: print('x_train = ', x_train.shape)
print('x_test = ', x_test.shape)
print('y_train = ', y_train.shape)
print('y_test = ', y_test.shape)

x_train = (105, 4)
x_test = (45, 4)
y_train = (105,)
y_test = (45,)
```

```
In [17]: import sklearn.metrics
from sklearn.tree import DecisionTreeClassifier

classifier = DecisionTreeClassifier()
classifier.fit(x_train, y_train)
predictions = classifier.predict(x_test)
confusion_array = sklearn.metrics.confusion_matrix(y_test, predictions)
print(confusion_array)

[[16  0  0]
 [ 0 17  1]
 [ 0  0 11]]
```

```
In [18]: print('TN : ', confusion_array[0,0])
print('FN : ', confusion_array[1,0])
print('TP : ', confusion_array[1,1])
print('FP : ', confusion_array[0,1])
```

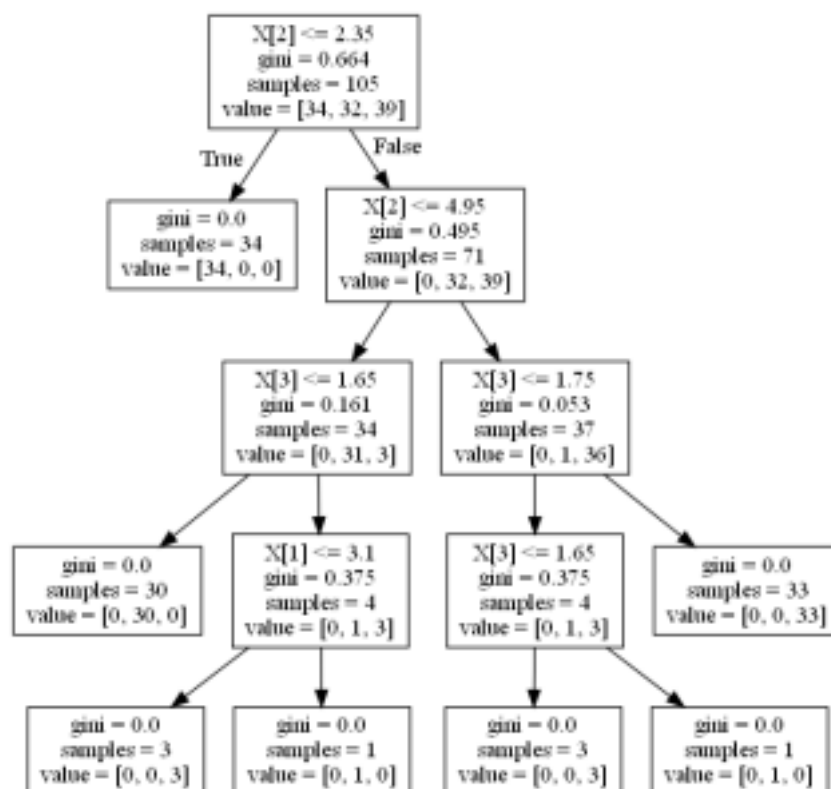
```
TN : 16
FN : 0
TP : 17
FP : 0
```

```
In [19]: sklearn.metrics.accuracy_score(y_test, predictions)
```

```
Out[19]: 0.9777777777777777
```

```
In [20]: from io import StringIO
from sklearn import tree
from IPython.display import Image
import os

os.environ['PATH'] += os.pathsep + 'C:/Program Files/Graphviz/bin/'
out = StringIO()
tree.export_graphviz(classifier, out_file=out)
import pydotplus
graph=pydotplus.graph_from_dot_data(out.getvalue())
Image(graph.create_png())
```



Latihan

1. Buat DataFrame panda 3x4 (3 baris x 4 kolom) yang kolomnya diberi nama **Zahra**, **Lily**, **Dito**, dan **Naufal**. Isi masing-masing dari 12 sel di DataFrame dengan bilangan bulat acak antara 0 dan 100.
2. Tampilkan output sebagai berikut:
 - Seluruh DataFrame
 - Nilai dalam sel baris #1 dari kolom Lily (Untuk kelas A), kolom Naufal(Untuk kelas B), dan kolom Zahra (Untuk kelas C).
3. Buat kolom kelima bernama Janet, yang diisi dengan jumlah baris dari kolom Zahradan Dito.
4. Cari sebuah kasus/permasalahan pada sebuah instansi/perusahaan, representasikan dalam bentuk decision tree.